

SkyQuery: Evaluating LLMs for Decision Making in the CAGE Challenge

Mark Bilinski

Naval Information Warfare Center
San Diego, CA

mark.bilinski.civ@us.navy.mil

Ryan Gabrys

Naval Information Warfare Center
San Diego, CA

ryan.c.gabrys.civ@us.navy.mil

Daniel Silva

Naval Information Warfare Center
San Diego, CA

daniel.silva61.civ@us.navy.mil

Abstract—This paper presents a novel framework for developing autonomous cyber defense agents that incorporate defender goals into their decision-making processes. Evaluated within the CAGE Challenge 2 simulation environment, the framework enables agents to adapt their strategies to align with specific objectives, such as maximizing rewards or deploying deception. Leveraging previously validated strategies, the proposed approach enhances the adaptability and effectiveness of cyber defense systems, providing a foundation for addressing evolving threats in dynamic cyber environments.

I. INTRODUCTION

The rise in the frequency and sophistication of cyberattacks has underscored the critical need for advanced defensive strategies capable of protecting enterprise networks. Autonomous Cyber Defense (ACD) systems have emerged as a promising solution, which uses artificial intelligence to counter adversarial threats in dynamic and complex environments. Within this domain, the Cyber Autonomy Gym for Experimentation (CAGE) Challenge 2 has become a benchmark environment to evaluate the performance of autonomous agents to defend enterprise networks against attackers [1].

The CAGE Challenge 2 environment, developed as part of the Cyber Operations Research Gym (CybORG) initiative, provides a simulation framework for post-exploitation scenarios. This environment includes a network comprising user, enterprise, and operational subnets, each of varying strategic importance. The challenge tasks Blue agents with defending these subnets from Red agents, which employ strategies such as reconnaissance, exploitation, and escalation. Key features of CAGE Challenge 2 include a diverse state-action space, configurable decoy mechanisms, and comprehensive scoring metrics to evaluate the effectiveness of defensive strategies. Enhancements to the original CybORG framework, as exemplified in CybORG++, have improved the scalability, transparency, and reliability of the simulation, enabling more robust experimentation with reinforcement learning-based defensive agents [2].

Despite the advancements made in the development of Blue agents for the CAGE Challenge 2 environment, existing methodologies often lack a systematic framework to guide their design and alignment with specific defensive goals. While efforts such as hierarchical reinforcement learning [4], ensemble methods, and heuristic strategies have shown promise, the

lack of a unified approach has limited the reproducibility and extensibility of these agents.

In this paper, we propose a novel framework for the systematic generation of Blue agents in the CAGE Challenge 2 environment. Our framework is designed with two primary principles in mind: (i) incorporating the goals of the defender into the decision-making process and (ii) grounding defensive strategies in previously vetted approaches that have shown effectiveness in past scenarios. This alignment ensures that the agents produced by our framework are both purposeful and adaptable, capable of addressing evolving threats and dynamic environments. Moreover, our framework can accommodate the evolution of defender goals over time, allowing seamless modification of strategies to meet shifting priorities or adversarial tactics.

Although our framework is validated within the context of the CAGE Challenge 2 environment, its design and principles may have broader applicability to real-world autonomous cyber defense. The ability to encode defender objectives and adapt to changing dynamics makes this approach particularly well suited for a wide range of cybersecurity challenges, including those beyond the scope of the CAGE Challenge 2 simulation. This work advances the state-of-the-art in autonomous cyber defense and establishes a replicable methodology for developing agents that align defensive strategies with organizational goals and evolving operational needs. Our contributions aim to bridge the gap between theoretical frameworks and practical implementation, fostering innovation in goal-driven, adaptive defense mechanisms.

II. RELATED WORKS AND CONTRIBUTIONS

The increasing complexity of cybersecurity threats has driven the development of advanced AI-driven frameworks to address evolving challenges. Among these, the concept of Large Language Model (LLM)-based agents has gained significant traction due to their ability to reason, plan, and interact with dynamic environments. Central to our work is the foundation laid by Moskal et al. [3], which demonstrates the transformative potential of LLMs in automating cyber threat testing and simulating adversarial activities. Their framework highlights the ability of LLM-supported agents to autonomously execute complex cyber campaigns, leveraging prompt engineering to navigate intricate decision-making

tasks. This approach serves as a basis for certain architectural elements in our proposed cyber defense framework.

The role of LLM agents in enhancing both offense and defense within cybersecurity environments is further explored in several studies. For instance, Rigaki et al. [9] introduced a modular cybersecurity environment where LLMs were evaluated in sequential decision-making tasks. Their experiments revealed that LLM agents could perform comparably or even surpass extensively trained traditional agents, emphasizing the adaptability of LLMs in handling complex, multi-agent scenarios. Similarly, An et al. [10] developed Nissist, a multi-agent system designed to generate mitigation plans based on historical incident data and troubleshooting guides. By significantly reducing response times and operational workload, this system highlights the practical utility of LLMs in formulating strategies.

Our work also draws inspiration from the orchestrated use of LLMs with external tools and specialized modules. Cui et al. [11] introduced LLMind, a framework that employs LLMs to coordinate complex tasks in IoT environments. By integrating domain-specific AI modules and employing finite state machines for task execution, LLMind demonstrated how modularity and adaptability enhance the effectiveness of autonomous agents. Similarly, Kahesh et al. [12] showcased Cyber Sentinel, a dialogue-based agent that interprets potential cyber threats and executes security actions in real time, improving operational efficiency and threat response capabilities.

In contrast to the offensive potential of LLMs, as demonstrated by Fang et al. [13] in their exploration of autonomous website hacking, our research focuses on leveraging LLMs to bolster defense. While these offensive capabilities illustrate the dual-edged nature of LLM technology, they also underscore the urgent need for robust, adaptive defensive frameworks like the one we propose. The contributions of this work are as follows: (i) We introduce a flexible framework, grounded in previously validated strategies, that integrates defender goals directly into the decision-making process, and (ii) We demonstrate, within the CAGE simulation environment, that the resulting agents exhibit behaviors consistent with the defender's specified goals.

III. CAGE CHALLENGE 2

CAGE Challenge 2 is a cybersecurity simulation framework designed to evaluate and compare the performance of defensive (blue) and offensive (red) agents within a networked environment. This challenge utilizes the CybORG platform, which provides a controlled environment to test autonomous cyber defense strategies.

In the CAGE Challenge 2 scenario, the blue agent is tasked with defending a manufacturing plant's network belonging to the nation of Florin, which is under cyber-attack from the rival nation Guilder. The network comprises three subnets: (i) Subnet 1: Contains non-critical user hosts, (ii) Subnet 2: Houses enterprise servers supporting user activities, (iii) Subnet 3: Includes critical operational servers essential for manufacturing processes.

The red agent begins with an initial foothold in subnet 1 and attempts to move laterally through the network to compromise the critical Operational Server in subnet 3. The blue agent's objective is to detect and mitigate these intrusions to maintain the integrity and availability of the operational systems.

The performance of the blue agent is evaluated based on a reward system that reflects the effectiveness of its defensive actions. More specifically, the blue reward when the red agent gains administrative access is -0.1 for low-importance systems and -1 for high importance systems. If red executes the impact action on the Operational Server then blue receives a reward of -10 . If blue performs a restore action, which requires that a system be brought offline for some duration of time, they are penalized -1 . The cumulative reward over the course of the scenario run determines the blue agent's final score. We refer to each scenario run as an episode and each episode consisted of 30 rounds where both blue and red execute one of their actions at every round.

The blue agent has a set of high-level actions to defend the network that include:

- Analyze: Conduct detailed analysis on hosts to detect malicious activities.
- Remove: Eliminate malicious software from hosts.
- Restore: Revert a host to a clean state from backup, ensuring removal of entrenched threats.
- Misinform: Deploy decoy services to deceive attackers and gather intelligence.

Each action has associated costs in terms of effort and potential system downtime, requiring the blue agent to balance these factors against the necessity of protecting various systems. For additional information into CAGE and the CybORG simulation environment, we refer the interested reader to [1].

In order to evaluate the effectiveness of different types of blue agents, two red agents were included as part of the challenge: (i) Meander Agent: Explores the network methodically, gaining privileged access on all hosts within a subnet before proceeding to the next, and (ii) B-Line Agent: Utilizes prior knowledge of the network layout to directly target the Operational Server.

IV. ARCHITECTURE AND PROMPT ENGINEERING

As discussed in the previous section, the blue agent in the CAGE simulation environment must select an action to execute during each round of the simulation. Our approach to enabling this functionality can be viewed as a variation of the LLM-centric Planner-Actor-Reporter framework originally introduced in [14]. This framework aligns with the OODA (Observe, Orient, Decide, Act) loop, guiding the agent's observation and action processes. Our instantiation of this framework cycles through the following two stages:

- 1) Action Retrieval: During this stage, the LLM accesses a set of datastores containing potential actions.
- 2) Action Selection: Given a set of potential actions along with some other additional information, the LLM is asked to select an action that can be executed.

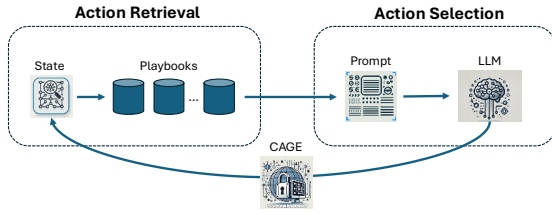


Fig. 1. Two Stage Agent Framework

This process is illustrated in more detail in Figure 1.

The agent executes in rounds where each round of the agent execution corresponds to a round of the CAGE scenario. The round begins with the action retrieval stage, which takes as input state information that is extracted from the CAGE simulation environment. The state information is then used to retrieve from a collection of cyber playbooks, which were developed from the existing submissions to the challenge, a set of actions that have been taken in previous games given a similar state. The output of this stage is a set of potential actions that will be combined into a prompt. In the next stage, the LLM selects from the collection of potential actions the action that best aligns with the defender’s stated goals and information about the previous observation from the game itself. Each round terminates with an action being selected and then executed in the CAGE simulation environment.

One of the challenges in designing our agent architecture stemmed from the stateless nature of the LLM itself. In particular, the LLM has no memory between the prompts provided as input, and consequently contextual information about prior interactions must be provided to the LLM at every round. Compounding this challenge is the fact that existing LLMs typically limit the sizes of the prompts to be around 4096 tokens (~3K words). In order to overcome these challenges, we made the following design decisions:

- 1) We initially developed a set of cyber playbooks that are based upon previous submissions to the CAGE Challenge 2 scenario. These playbooks allow us to focus the attention mechanisms of the LLM in the second stage on existing policies. By recommending potential actions, we are able to reduce some of contextual information required in the second stage since this information is already previously used to retrieve information in the first stage. Under our setup, in the second stage the agent simply needs to decide between previously vetted actions/policies.
- 2) We carefully construct our prompts following the general template in [3]. This format is similar in spirit to a single round of the well-known Reasoning and Action (ReAct) framework, which has received significant attention in the past [15].

Next, we turn our attention to discussing in more detail the design of the two stages of our agent architecture.

A. Action Retrieval Stage

The key architectural component of the action retrieval stage is the agent datastore. Our aim was to first construct a capability that mimics the behavior of an individual agent submitted to the CAGE Challenge. For shorthand let X denote an agent submitted to CAGE. Our approach was to first observe for a certain interval of time (in this case 30 rounds) the behavior of each agent across a series of simulations. For each round $r \in [30] = \{1, \dots, 30\}$, we recorded an individual memory where each memory consisted of the following two pieces of information: (i) The action taken at round r , and (ii) The previous m observations where $m \in \{1, 2, 3, 4, 5\}$.

This information is then used to generate a json record that encapsulates how the agent behaved on average given a sequence of m previous observations. For this particular setting, the m previous observations are referred to as the state. Figure 2 is displaying an example illustrating a record extracted from the Mindrake agent datastore. The state information consists of the first nine lines of $m = 3$ previous observations. In this case, we see that given the previous observations (displayed as the state information Figure 2), approximately 63% of the time, the Mindrake agent will perform the action “Remove User3.” The other action that may be taken is “Remove Op_Server0,” which given the current state was observed to occur approximately 37% of the time during the course of our simulations.

```

{ "state": "
Hostname User2 has activity Scan
where the following has been compromised
Unknown.
Hostname User2 has activity Exploit where
the following has been compromised
User.
Hostname User2 has no activity where the
following has been compromised Unknown.,

"actions": [
{ "action": "Remove User3",
  "p": 0.63,
  "stats": { "infiltration_penalty": 27.89,
    "failed_exploits": 8.17,
    "scanned_hosts": 12.98,
    "successful_exploits": 7.69,
    "mean_reward": -6.31
  }
},
{ "action": "Remove Op_Server0",
  "p": 0.37,
  "stats": { "infiltration_penalty": 20.12,
    "failed_exploits": 5.89,
    "scanned_hosts": 12.85,
    "successful_exploits": 3.94,
    "mean_reward": -8.72 } } ] }

```

Fig. 2. Datastore Agent Record

In addition to the actions associated with specific states, note in Figure 2 that each action includes a set of statistics. These statistics reflect the average score the Mindrake agent is expected to achieve at the end of the game when it takes that specific action in the given state. Following this example, Figure 2 states that given the current state if the action “Remove Op_Server0” is chosen, then at the end of the game, the expected infiltration penalty is 20.12, the expected number of failed exploits is 5.89, the expected number of scanned hosts is 12.85, the expected number of successful exploits is 3.94, and the expected mean reward is -8.72 .

We created a set of datastores, with each record containing the data illustrated in Figure 2. To generate the memory datasets for these datastores, we performed simulations for each agent X (where X represents Cardiffuni, Mindrake, Keepin’ it RL - Ensemble, or Keepin’ it RL - Transfer Learning). Each agent was first tested against the B-line red agent for 8,000 episodes, each consisting of 30 rounds. Following

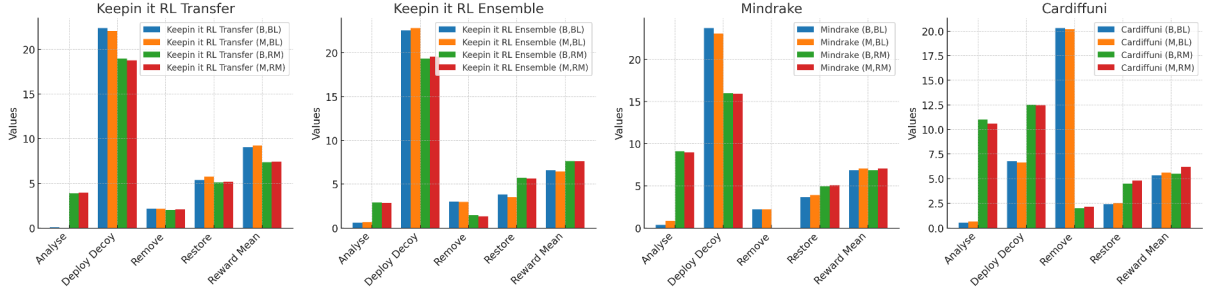


Fig. 3. Comparison of Baseline and Mimicry Agents

this, the same agent was run against the Meander red agent for an additional 8,000 episodes of 30 rounds each. The memories gathered from these simulations were compiled into a single datastore for each agent, representing their respective behavioral information. This process resulted in four distinct datastores, where Cardiffuni's state representation included $m = 5$ previous observations, Mindrake's state used $m = 3$ previous observations, and both Keepin' it RL agents relied on $m = 1$ previous observation for their state representations.

One of the surprising findings we confirmed during our process was that the datastores appeared to encapsulate all the essential information needed to replicate the behavior of each agent against both B-line and Meander opponents. To demonstrate this, we created a set of four "mimicry" agents—one for each of Cardiffuni, Mindrake, Keepin' it RL - Ensemble, and Keepin' it RL - Transfer. For each agent X and a given state, we utilized LlamaIndex [5] to store and retrieve the most similar states encountered previously. The retrieved information was used to determine the actions previously taken in similar states, enabling the mimicry agents to replicate the behavior of the original agents. The embeddings for state representation were generated using SecBERT, a BERT-based model pre-trained on cybersecurity-specific text, integrated with the HuggingFace Transformers library [6]. The similarity between states was calculated using the cosine distance metric.

When a datastore agent record is retrieved, the system accesses the actions stored in its metadata. The next action is then determined by sampling from the categorical probability distribution associated with the corresponding datastore record. For example, if the record shown in Figure 2 is retrieved for a given state, there is a 63% probability that after sampling the resulting action "Remove User3" will be selected.

Figure 3 presents execution statistics for each mimicry agent compared to its corresponding original agent over 30 rounds against both Meander and B-line opponents. The notation is defined as follows: (i) (B, BL): The original (baseline) agent played against B-line. (ii) (M, BL): The mimicry agent played against B-line. (iii) (B, RM): The original (baseline) agent played against Meander. (iv) (M, RM): The mimicry agent played against Meander. Each plot displays the average statistics calculated from 500 episodes, with each episode

consisting of 30 rounds. The "Reward Mean" represents the negative value of blue's average reward, while the other data points indicate the average number of times per game that blue performed specific actions. For instance, in Figure 3, the mimicry Cardiffuni agent executed the "Remove" action approximately twice on average during 30 rounds in each game against Meander.

A key observation is that while our mimicry agents demonstrated behavior closely aligned with their original counterparts in terms of the gathered statistics, there were significant differences in behavior across the various mimicry agents. For example, Cardiffuni executed the "Remove" command an average of about two times every 30 rounds against Meander, whereas Mindrake almost never used the "Remove" command. This diversity in behavior among the agents is crucial and will be leveraged to generate a range of behavioral responses tailored to the defender's objectives.

The 4 actions that are produced at the end of the Action Retrieval stage and subsequently used to generate our prompts in the Action Selection Stage are the four actions chosen by each of the mimicry agents represented in Figure 3.

B. Action Selection Stage

For the action selection stage, we utilized a standardized template for crafting prompts to elicit responses from the LLM. Following the approach outlined in [3], each prompt can be broken down into three key components:

- 1) **Setup:** Provides a high-level overview of the environment and the objective the LLM is tasked with achieving.
- 2) **Instruction:** Clearly specifies the desired output, including the format in which the LLM should deliver its response.
- 3) **Context:** Supplies relevant background information that the LLM can use to accurately generate responses based on the instructions.

This structured approach ensures consistency and clarity in the LLM's interaction and decision-making process.

An example of the setup, instruction, and context sub-prompts is shown in Figure 4. Note that, for illustration purposes, the context sub-prompt in this figure displays only a single action. However, the results presented in the next

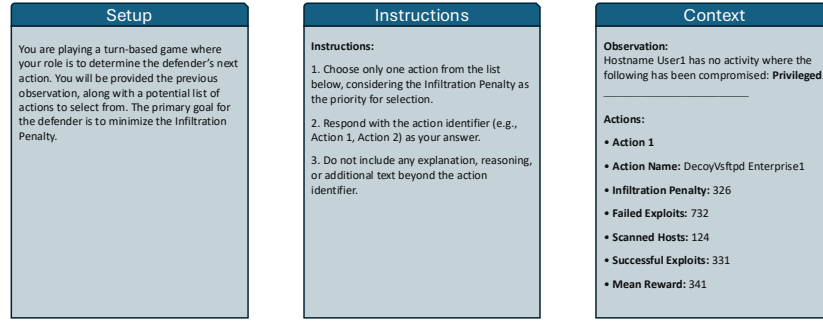


Fig. 4. Action Selection Stage Prompt.

section are based on prompts that included four actions for the LLM to choose from, with each action drawn from one of the four datastores constructed as described in Section IV-A. The complete prompt, which is provided as input to the LLM and depicted in Figure 1, is created by concatenating the setup sub-prompt, the instruction sub-prompt, and the context sub-prompt in that order. These combined prompts were then processed by the Llama 3.1 8b instruct model.

Notice in Figure 4 that in addition to specifying potential actions to take, the context sub-prompt also enumerates the meta-data statistics originally contained in our datastore agent records. Specifically, for a given action we report on the following: (i) the number of compromised hosts which appears as the Infiltration Penalty, (ii) the total number of failed exploits, (iii) the number of scanned hosts, (iv) the number of successful exploits, and (v) the mean reward.

V. RESULTS

In this section, we evaluate the performance of the LLM agent architecture described in the previous section within the CAGE simulation environment. Specifically, we examine the behavior of five LLM agents, each distinguished by a variation in the final sentence of the setup sub-prompt from Section IV-B (see Figure 4), which begins with the phrase, “The primary goal for the defender is to ...”. The variations are as follows:

- 1) **No Goal:** In this configuration, no specific goal is provided to the model in the setup sub-prompt. The sentence beginning with “The primary goal...” (see Figure 4) is omitted entirely.
- 2) **Minimize Restore:** “The primary goal for the defender is to minimize the use of Restore.”
- 3) **Minimize Infiltration Penalty:** “The primary goal for the defender is to minimize the Infiltration Penalty.”
- 4) **Maximize Reward:** “The primary goal for the defender is to maximize the Reward Mean.”
- 5) **Maximize Deception:** “The primary goal for the defender is to maximize the use of deception.”

Notice that the Minimize Restore and Maximize Deception goals are directly the result of actions that are taken by blue whereas the Maximize Reward and Minimize Infiltration Penalty depend on the actions of both the red and blue agent in the CAGE simulation environment.

To better understand the behaviors exhibited by our agents, we collected and analyzed statistics from evaluations of all five LLM agents against both Meander and B-line. Specifically, we considered: (i) Reward Mean: The average reward per game, presented as the negative of blue’s reward for consistency with the other metrics, as in Figure 3. (ii) Num Decoys: The average number of decoys deployed per episode throughout the simulation. (iii) Num Compromised Hosts: The average number of actively compromised hosts per episode. (iv) Restore: The average number of restore calls made per episode.

The results, based on running each of the five LLM agents for 500 episodes of 30 rounds each in the CAGE simulation environment, are displayed in Figure 5. Notice that there is considerable variation between the different types of agents and how they perform against Meander and B-line. For instance, Maximize Deception deployed nearly 26 decoys per round against Meander and almost 23 per round against B-line, whereas Maximize Reward deployed less than 13 against B-line and around 7 against Meander.

In general, the behavior of the agents was consistent with the stated goal. For instance, against both B-Line and Meander, the Maximize Deception agent deployed significantly more decoys on average than the other agents under consideration. In addition, the Maximize Reward agent consistently achieved the best possible reward, although note that the reward in this case was still less than the Cardiffuni mimicry agent discussed in Figure 3. One of the possible reasons for this disparity is likely due to the probabilistic nature of the model itself and in particular due to the fact that a small fraction of the time the model chose an action that did not necessarily align with the stated goal in the setup sub-prompt, and in this case led to the occasional selection of actions with lower rewards.

One of the surprising outcomes that arose from our simulations is that not only did our LLM agents behave in accordance with the provided goals for the goals that depended on blue actions only, but the LLM agents also achieved goals that depend upon **both** blue and red actions (i.e., the maximize reward and minimize infiltration penalty goals).

As for performance against Meander and B-line specifically, many of the agents were comparable but had a few notable differences. The starkest difference was in minimize infiltration penalty agent’s reward mean being near best against B-

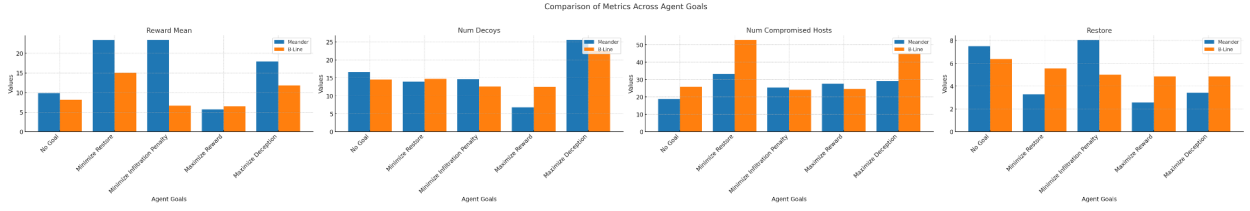


Fig. 5. LLM Agents vs. Red Agents

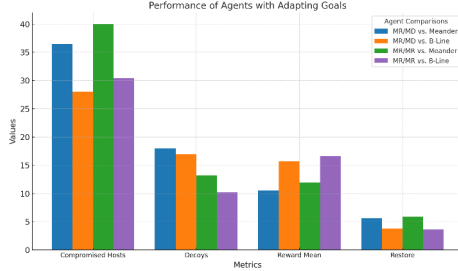


Fig. 6. LLM Agent Performance, Non-Static Goals

line and worst with meander. Across the other metrics, that agent was comparable except with a similar disparity on use of Restore. This suggests Restore is much more effective against B-line, and in particular more effective at keeping the number of compromised hosts low – which makes sense since in general B-line tends to target much fewer hosts than Meander. Incidentally, the maximize reward strategy does the opposite and makes more frequent calls to Restore when playing against B-line vs. Meander further reinforcing that observation.

Lastly, we note that when the LLM agent was provided no goal, it performed surprisingly well with respect to many of the statistics collected. In terms of reward, it was the second highest scoring agent when evaluated against B-Line and the third highest against Meander. Another area where this particular agent performed well was in terms of minimizing the number of compromised hosts. Against B-Line, the No Goal LLM Agent had on average the lowest number of compromised hosts but at the cost of calling the restore command more frequently than the majority of the other agents.

Next, we consider the performance of our LLM agent in scenarios where the defender's objective may shift during an episode. Figure 6 presents statistics for two LLM agents, each simulated for 500 episodes of 30 rounds:

- 1) **Maximize Reward/Maximize Deception (MR/MD)**: At each round, we randomly select one of two sub-prompts with equal probability, specifying whether the defender's objective is to maximize reward or maximize deception.
- 2) **Maximize Reward/Minimize Restore (MR/MR)**: At each round, we randomly select one of two sub-prompts with equal probability, specifying whether the defender's objective is to maximize the reward or minimize restore.

Figure 6 reveals that, overall, the MR/MD agents deployed more decoys compared to the MR/MR agents, while the

MR/MR agents did not make fewer restore calls. Interestingly, prioritizing reward maximization and the use of deception resulted in a better overall mean reward. This suggests that, in this game, employing deception may be a more effective strategy than minimizing restore actions.

VI. CONCLUSION

In this work, we presented a flexible framework for developing autonomous cyber defense agents that align their behavior with specific defender goals. Through evaluation in the CAGE Challenge 2 environment, we demonstrated that the agents effectively adapt their strategies, such as maximizing rewards or deploying deception, to meet these goals. This provides a foundation for future research into adaptive, goal-driven cyber defense systems capable of addressing evolving threats.

REFERENCES

- [1] M. Standen et al., "Cyber autonomy gym for experimentation challenge 2," <https://github.com/cage-challenge/cage-challenge-2>, accessed Dec. 2024.
- [2] H. Emerson, et al., "CybORG++: An Enhanced Gym for the Development of Autonomous Cyber Agents," arXiv:2410.16324, 2024.
- [3] S. Moskal, S. Laney, E. Hemberg, and U.M. O'Reilly, "LLMs killed the script kiddie: How agents supported by large language models change the landscape of network threat testing," arXiv:2310.06936, 2023.
- [4] A.V. Singh et al., "Hierarchical Multi-agent Reinforcement Learning for Cyber Network Defense," arXiv:2410.17351, 2024.
- [5] LlamaIndex. (2024). *LlamaIndex: Efficient Document Retrieval and Embedding Integration*. [Online]. Available: <https://www.llamaindex.com/docs>. [Accessed: Dec. 6, 2024].
- [6] HuggingFace, "Transformers: State-of-the-art Natural Language Processing for Pytorch and TensorFlow 2.0," *HuggingFace Transformers Documentation*, 2024. [Online]. Available: <https://huggingface.co/transformers>. [Accessed: Dec. 6, 2024].
- [7] Y. Zhang et al., "SecBERT: A cybersecurity-focused BERT model for enhanced text representation," *Proceedings of the 2024 International Conference on Cybersecurity AI*, pp. 30-45, May 2024.
- [8] N. Jeffrey, et al., "Using Ensemble Learning for Anomaly Detection in Cyber-Physical Systems," *Electronics*, vol. 13, pp. 1391, Apr. 2024.
- [9] M. Rigaki, O. Lukas, C.A. Catania, and S. García, "Out of the cage: How stochastic parrots win in cyber security environments," *Proceedings of the 16th International Conference on Agents and Artificial Intelligence*, vol. 3, Rome, Italy, pp. 774–781, 2024.
- [10] K. An et al., "Nissist: An incident mitigation copilot based on troubleshooting guides," *European Conference on Artificial Intelligence*, Santiago de Compostela, Spain, 2024.
- [11] H. Cui, et al., "LLMind: Orchestrating AI and IOT with LLM for complex task execution," arXiv:2312.09007, 2024.
- [12] M. Kaheh, et al., "Cyber sentinel: Exploring conversational agents in streamlining security tasks with GPT-4," arXiv:2309.16422, 2023.
- [13] R., Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "LLM agents can autonomously hack websites," arXiv:2402.06664, 2024.
- [14] I. Dasgupta et al., "Collaborating with language models for embodied reasoning," arXiv:2302.00763, 2023.
- [15] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.